# Assignment 1: Design and Implementation of Data Tables and their Conceptual Designs

## Project Title: StayEase – Hostel Management System

**Team: QueryCraft**

Aryan Kumar (24110055)
Niraj Kumar (24110222)
Nityanand Karmali (24110226)
Parth Kale (24110242)
Patil Nachiket Kiran (24110250)

---

## ER Diagram:

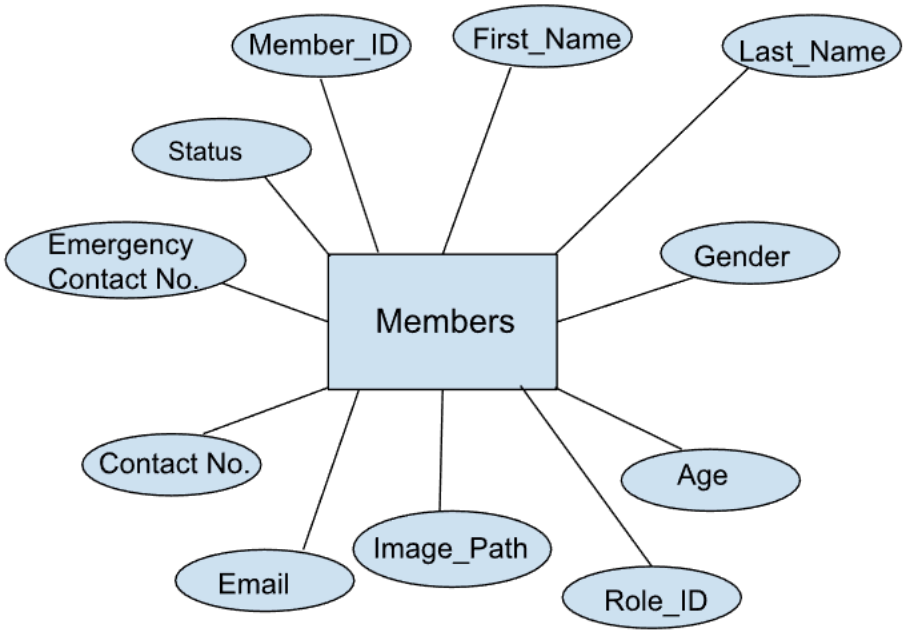


Figure 1: ER Diagram of the Members table only

The attributes of the entity can be represented like this in an ER Diagram, but due to space constraints and for better interpretability, we just wrote all the attributes in the entity box only.

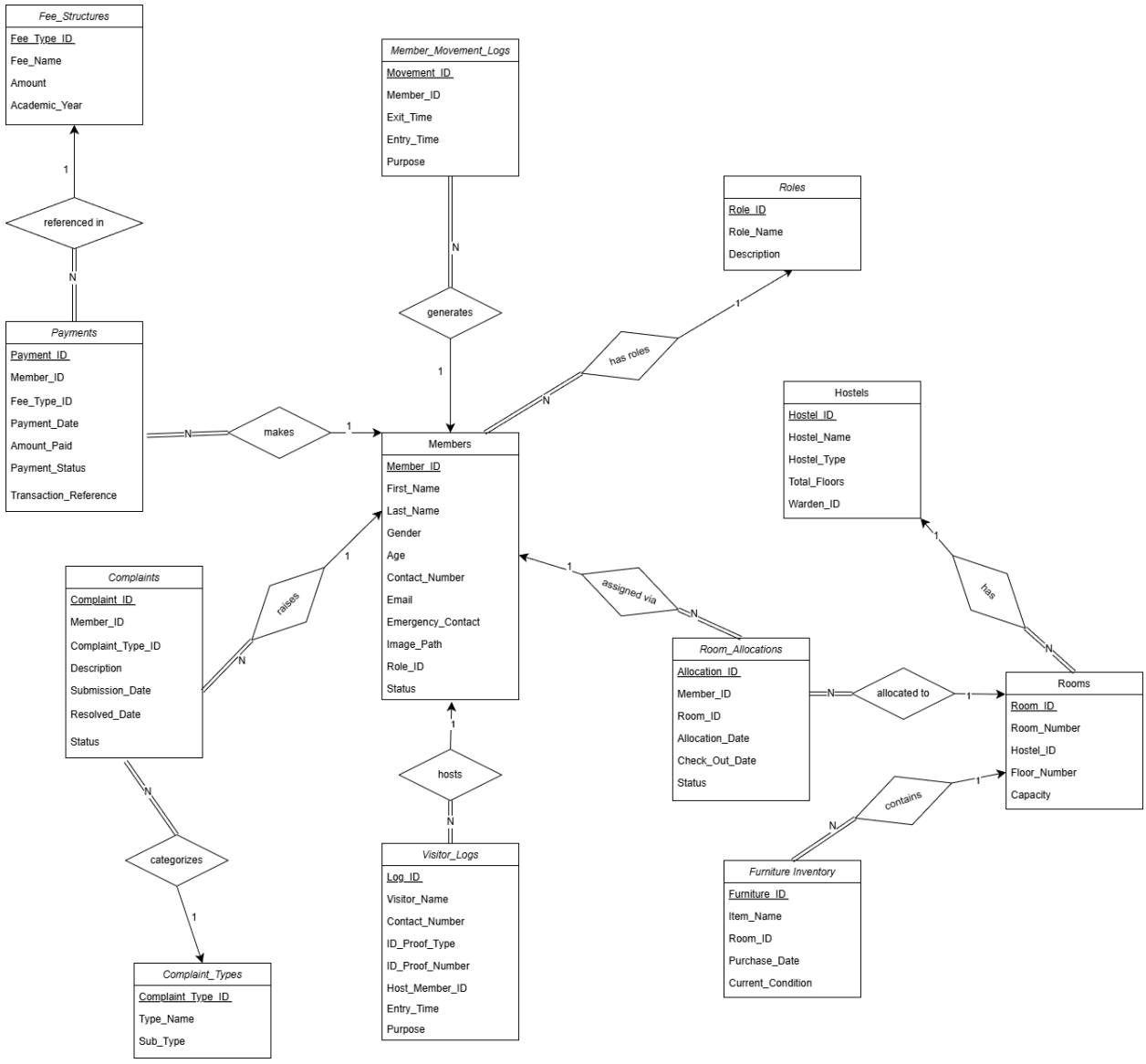


Figure 2: ER diagram of StayEase database

## StayEase – Relationship Justifications:

1. **Roles – Members (1: N)**

   **Relationship:** has roles

   **Relationship Type:** One-to-Many

   **Justification:**

   Each member of the hostel must belong to exactly one system role (e.g., Student, Warden, Technician). However, a single role can be assigned to multiple members. For example, many members can have the role ”Student,” but each student has only one role at a time. Therefore, the relationship between Roles and Members is one-to-many.

2. **Hostels – Rooms (1: N)**

   **Relationship:** has

   **Relationship Type:** One-to-Many

   **Justification:**

   A hostel building contains multiple rooms, but each room belongs to exactly one hostel. This reflects real-world structure, where a room cannot exist independently of a hostel. Therefore, the relationship between Hostels and Rooms is one-to-many. It also ensures structural hierarchy and proper room organization.

3. **Members – Room Allocations (1: N)**

   **Relationship:** assigned via

   **Relationship Type:** One-to-Many

   **Justification:**

   A member may change rooms over time, resulting in multiple room allocation records. However, each allocation record corresponds to only one member. Hence, the relationship between Members and Room Allocations is one-to-many. This design supports historical tracking of room changes while maintaining allocation integrity.

4. **Rooms – Room Allocations (1: N)**

   **Relationship:** allocated to

   **Relationship Type:** One-to-Many

   **Justification:**

A room can be allocated to different members across different time periods. Each allocation record refers to exactly one room. Therefore, the relationship between Rooms and Room Allocations is one-to-many.

Room Allocations acts as an associative entity resolving the many-to-many relationship between Members and Rooms.

5. **Rooms – Furniture Inventory (1 N)**

**Relationship:** contains

**Relationship Type:** One-to-Many

**Justification:**

Each room can contain multiple furniture items such as beds, chairs, and tables. However, each furniture item belongs to only one room at a time. Hence, the relationship is one-to-many. This enables tracking of hostel assets and their condition.

6. **Members – Visitor Logs (1: N)**

**Relationship:** hosts

**Relationship Type:** One-to-Many

**Justification:**

A member may receive multiple visitors. Each visitor log entry corresponds to exactly one host member. Therefore, the relationship between Members and Visitor Logs is one-to-many. This ensures proper security tracking and accountability for visitor entries.

7. **Members – Member Movement Logs (1: N)**

**Relationship:** generates

**Relationship Type:** One-to-Many

**Justification:**

A member may exit and enter the hostel multiple times. Each movement record belongs to exactly one member. Therefore, the relationship is one-to-many. This helps monitor student movement and enforce time-based rules.

8. **Members – Payments (1: N)**

**Relationship:** makes

**Relationship Type:** One-to-Many

**Justification:**

A member can make multiple payments for different fee types or semesters. Each payment record corresponds to only one member. Therefore, the relationship is one-to-many. This ensures accurate financial tracking per member.

9. **Fee Structures – Payments (1: N)**

**Relationship:** referenced in

**Relationship Type:** One-to-Many

**Justification:**

A fee structure (e.g., Hostel Rent) may generate multiple payments from different members. However, each payment refers to one specific fee type. Therefore, the relationship is one-to-many. This separates fee definition from payment transactions.

10. **Members – Complaints (1: N)**

**Relationship:** raises

**Relationship Type:** One-to-Many

**Justification:**

A member may raise multiple complaints over time. Each complaint is filed by exactly one member. Hence, the relationship is one-to-many. This ensures accountability and complaint tracking.

11. **Complaint Types – Complaints (1: N)**

**Relationship:** categorizes

**Relationship Type:** One-to-Many

**Justification:**

Each complaint belongs to a specific complaint type (e.g., Electrical → Fan). A complaint type can classify many complaints. Therefore, the relationship is one-to-many.

This improves categorization and maintenance management.

## Additional Constraints:

1. **Domain Constraints**
   - Age must be greater than 0.

- Room capacity must be greater than 0.
- Payment amount must be greater than 0.
- Gender is restricted to predefined values.
- Status fields are restricted to predefined sets.
- The Current_Condition of furniture is restricted to valid states.
- Payment Status is restricted to 'Success' or 'Failed'.

2. **Temporal Constraints**
   - Check_Out Date must be greater than Allocation_Date.
   - Entry Time must be greater than Exit Time for Member Movement Logs table(or NULL when not returned).
   - If Complaint Status = 'Resolved', then Resolved_Date must not be NULL.

   These constraints ensure logical consistency in time-based records.

3. **Uniqueness Constraints**
   - Email must be unique.
   - Contact Number must be unique.
   - The Transaction Reference must be unique.
   - Room_Number must be unique within each hostel.

   These prevent duplication and ensure the unique identification of records.

4. **Referential Integrity Constraints**

   Foreign keys enforce relational consistency:
   - Members must reference a valid Role.
   - Rooms must reference a valid Hostel.
   - Allocations must reference valid Members and Rooms.
   - Payments must reference valid Members and Fee Structures.
   - Complaints must reference valid Members and Complaint Types.
   - Appropriate ON DELETE and ON UPDATE rules were used to prevent orphan records.

5. **Business Logic Constraints**
   - A member can have only one active room allocation at a time (enforced logically).
   - Visitor logs must always reference a valid host member.
   - Room occupancy is derived from active allocations rather than being stored redundantly.

## Contributions:

1. **Nachiket Patil**: Helped in brainstorming of the database, features, etc, and also created a SQL dump file.
2. **Niraj Kumar**: Documented the assignment and helped in brainstorming the database and relationships.
3. **Nityanand Karmali**: Reviewed and refined ER diagrams and also helped with MYSQL codes for table creation and constraints
4. **Parth Kale**: Helped in the structuring of tables, PK, FK, etc, and the relationship for better interpretation of the database.
5. **Aryan Kumar**: Created an ER diagram and also suggested final relationships to incorporate in the database